

TDD Manifesto

Test Driven Development in every home

MENU



Getting started

Why test first

If we want to trust our code, we need to be able to trust our tests. If we want to trust our tests we need to be able to see them failing. If we never see our tests fail we don't know if our tests are actually testing the right things. Seeing a test failing is as important as seeing it passing. A test failure validates that the test is meaningful and unique. It's a software expectation to fulfill. It gives us a target to aim for, showing that the code we are about to implement is useful and necessary.

The essence of TDD

The rules of Test Driven Development are relatively simple. It is a technique we can already benefit from in the first two weeks and we can still learn from it after twenty years. TDD forces us to carry out a set of practices that will all help us build quality software.

It will force us to:

- work in short cycles
- write tested code
- do continuous refactoring
- continuously analyze the requirements (hypotheses)
- producing minimalist and not-goldplated code

When we can't use TDD

We can not use TDD if:

- we don't have clear expectations for the code we are about to implement
- testing does not make sense or does not bring any benefit
- the tests run very slow, allowing only long feedback loops
- the design requires visual verification such as UI layout or UX concerns

Steps to get started

1. Familiarize yourself with the three laws of TDD

The fundamentals of TDD consist of three main rules. These rules constrain us to change only one thing at a time. They force us to apply baby steps!

TDD is all about the **frequency of feedback**. It lets us fail only a little with a small cognitive overhead, where the changes are cheap. It helps us to quickly recover from mistakes, and most importantly to avoid any big mistakes in the code.

First of all, familiarize yourself with the three laws of TDD! In order to practice TDD efficiently, we should strictly follow these three fundamental rules:

1. You are not allowed to write any production code unless it is to make a failing unit test pass.
2. You are not allowed to write any more of a unit test than is sufficient to fail, and compilation failures are failures.
3. You are not allowed to write any more production code than is sufficient to pass the one failing unit test.

2. Learn the Red-Green-Refactor TDD cycle

To practice TDD, we need to go through the TDD Red-Green-Refactor cycle, repeatedly in iterations. This loop will ensure that we always take small and verified steps when writing code, resulting in short feedback loops.

Red phase – Write a failing test: We have to write a failing test based on a specific requirement. Here we must follow the second TDD rule: we can not write more than one unitary failing test.

Green phase – Make the test pass: Our task is to make the failing test pass by writing production code. Here we also look after the third rule of TDD: We are not allowed to write more than sufficient production code.

Refactor phase – Refactoring: We need to practice refactoring and following best clean code practices. We need to use proper namings, extract logic into proper abstractions, and simplify code. Once we make the refactoring, we need to start over the TDD cycle with the red phase. We should never refactor in other phases, only here.

3. Start writing code with TDD

3.1. Get started with Kata exercises

After you familiarized yourself with the main rules and flow of TDD, it is time to put the learnings into practice. We have collected some great exercises for you to efficiently get started with TDD. Please visit the **Exercises section**.

3.2 Apply TDD in your project

Once you finished with solving several Katas, it's time to put TDD in practice in your project. Start **small!** Start with a **failing test** for a function or a class you want to implement! Follow the **3-phase** design! And

always keep the **three laws of TDD** in mind!

4. Level up your TDD skills

You can start applying TDD without any deep technical knowledge, but we recommend investing time in learning these topics to benefit the most from TDD:

- Writing tests: how to write unit tests in the desired language/framework, including mocking
- Writing clean code: how to write well-designed code by using best practices, such as using proper namings or applying useful design patterns
- Writing clean tests: how to write clean and maintainable tests. For more info about it, please visit our [A clean test](#) section.